

Programación para Física y Astronomía

Departamento de Física.

Coordinadora: C Loyola

Profesores C Femenías / D Basantes

Primer Semestre 2025

Universidad Andrés Bello

Departamento de Física y Astronomía



Introducción y Contexto

Fundamentos de Pandas para Análisis de Datos

Fundamentos de Pandas

Manejo de datos con Pandas

Manejo avanzado de datos con pandas

Limpieza de datos

Lectura y Exportación de Datos

Visualización de Datos con Pandas

Ejercicios Guiados: Impactos de Meteoritos

Introducción y Contexto

- **Semana 7, Sesión 1 (Sesión 13):**
 - Exploramos gráficos avanzados en Matplotlib (subplots múltiples, histogramas, 3D).
 - Tuvimos un vistazo opcional a pandas para el manejo de datos tabulares.
- **Semana 7, Sesión 2 (Sesión 14):**
 - Realizamos un **problema evaluado** integrando NumPy y Matplotlib (matriz aleatoria, álgebra lineal, gráficas 3D e histograma).
 - Aclaramos dudas tras la entrega.
- **Objetivo de hoy:** Iniciar **programación orientada a objetos (POO)** en Python y repasar análisis básico de datos en NumPy/pandas.

- **Comprender** los fundamentos de la Programación Orientada a Objetos (POO) en Python.
- **Explorar** el uso de **pandas** para análisis y manipulación de datos.
- **Aplicar** operaciones básicas y avanzadas de pandas en contextos físicos y astronómicos.
- **Desarrollar** habilidades para combinar POO y pandas en problemas prácticos.

Fundamentos de Pandas para Análisis de Datos

Fundamentos de Pandas para Análisis de Datos $\ni$ ¿Por qué Pandas para Análisis de Datos?

- **DataFrames:** estructuras tabulares similares a Excel/hojas de cálculo
- **Lectura:** archivos CSV, Excel, JSON automáticamente
- **Operaciones:** filtrado, agrupación, estadísticas descriptivas
- **Limpieza:** manejo de datos faltantes, duplicados
- **Integración:** funciona perfectamente con NumPy y Matplotlib

Para Física/Astronomía

Ideal para analizar datos experimentales, observaciones astronómicas, resultados de simulaciones.

Fundamentos de Pandas para Análisis de Datos $\ni$ ¿Qué es pandas?

- **pandas** es una biblioteca de Python para análisis y manipulación eficiente de datos.
- Permite trabajar con datos tabulares (tipo Excel) de forma sencilla y poderosa.
- Ofrece estructuras clave: **Series** (vectores) y **DataFrame** (tablas).
- Esencial para ciencia de datos, física computacional y astronomía.

Ventajas

- Lectura y escritura de archivos (CSV, Excel, etc.)
- Filtrado, selección y limpieza de datos
- Estadísticas y agrupaciones rápidas
- Integración con NumPy y Matplotlib

Fundamentos de Pandas

Fundamentos de Pandas $\ni$ Series: Vector unidimensional

```
1 import pandas as pd
2
3 # Series
4 s = pd.Series([1.5, 2.0, 0.8])
5
6 s
```

# 0	
0	1.5
1	2.0
2	0.8

- **Series:** vector unidimensional con etiquetas de índice.

```
1 import pandas as pd
2
3 # Series
4 s = pd.Series([1.5, 2.0, 0.8], index=['objeto1',
   ↪ 'objeto2', 'objeto3'])
5
6 s
```

# 0	
objeto1	1.5
objeto2	2.0
objeto3	0.8

Discusión: Una **Series** es como un vector con índice.

Fundamentos de Pandas $\ni$ DataFrame: Tabla bidimensional

- **DataFrame**: tabla bidimensional con filas y columnas etiquetadas.

```
1 import pandas as pd
2
3 # DataFrame
4 df = pd.DataFrame({
5     'masa': [1.5, 2.0, 0.8],
6     'velocidad': [10.5, 8.2, 15.3]
7 })
8
9 df
```

	# masa	# velocidad
0	1.5	10.5
1	2.0	8.2
2	0.8	15.3

Discusión: Un DataFrame es como una tabla de Excel.

Fundamentos de Pandas $\ni$ Etiquetas personalizadas en DataFrame

- **DataFrame**: tabla bidimensional con filas y columnas etiquetadas.

```
1 df.rename(index={3: "Fila 3"}, inplace=True)
2
3 df
```

	# masa		# velocidad
1		1.5	10.5
2		2.0	8.2
Fila 3		0.8	15.3

Discusión: Se pueden personalizar las etiquetas de filas y columnas.

Fundamentos de Pandas $\ni$ Acceso y selección de datos en DataFrame

```
1 # Acceso por columna
2 df['masa']
```

# masa	
0	1.5
1	2.0
2	0.8

Discusión: Este ejemplo muestra cómo acceder a una columna específica de un DataFrame utilizando su nombre como clave. Es útil para realizar operaciones o análisis sobre una sola variable.

```
1 # Acceso por fila (iloc: por posición, loc: por
  ↳ etiqueta)
2 df.iloc[2] # Tercera fila
3 df.loc["Fila 3"] # Fila con nombre "Fila 3"
```

# Fila 3	
masa	0.8
velocidad	15.3

Discusión: Aquí se muestra cómo acceder a filas específicas de un DataFrame, ya sea por su posición numérica ('iloc') o por su etiqueta ('loc'). Esto es útil para inspeccionar o modificar datos en filas específicas.

Fundamentos de Pandas $\ni$ Operaciones vectorizadas y estadísticas

```
1 # Operaciones sobre columnas
2 df['energia_cinetica'] = 0.5 * df['masa'] * df['velocidad']**2
3
4 df
```

	# masa	# velocidad	# energia_cinetica
1	1.5	10.5	82.6875
2	2.0	8.2	67.24
Fila 3	0.8	15.3	93.63600000000002

Discusión: Las operaciones se aplican a toda la columna sin necesidad de bucles explícitos.

Fundamentos de Pandas $\ni$ Estadísticas descriptivas rápidas

```
1 # Operaciones sobre columnas
2 # Estadísticas rápidas
3 df['energia_cinetica'].mean() # Promedio de energía cinética
4 # Output: 81.18783333333334
5 df.describe()
```

	# masa	# velocidad	# energia_cinetica
count	3.0	3.0	3.0
mean	1.4333333333333333	11.333333333333334	81.18783333333334
std	0.6027713773341707	3.6226141573915016	13.261747776342819
min	0.8	8.2	67.24
25%	1.15	9.35	74.96375
50%	1.5	10.5	82.6875
75%	1.75	12.9	88.16175000000001
max	2.0	15.3	93.63600000000002

Discusión: Las estadísticas descriptivas proporcionan una visión general de la distribución y características de los datos en el DataFrame.

Fundamentos de Pandas $\ni$ Limpieza y manejo de datos faltantes

```
1 import numpy as np
2 df.loc[1, 'velocidad'] = np.nan # 0 cualquier valor que uno quiera ingresar
3
4 df
```

	# masa	# velocidad	# energia_cinetica
1		1.5	Missing value
2		2.0	8.2
Fila 3		0.8	15.3

```
1 df_clean = df.dropna()
2
3 df_clean
```

	# masa	# velocidad	# energia_cinetica
2		2.0	8.2
Fila 3		0.8	15.3

Discusión: pandas facilita la detección y el tratamiento de datos incompletos, común en experimentos reales.

Manejo de datos con Pandas

Manejo de datos con Pandas $\ni$ Nuevos datos de ejemplo para agrupación

```
1 import random
2 import pandas as pd
3 # Creamos un DataFrame de ejemplo con datos aleatorios con 100 filas
4 data = {
5     'masa': [random.uniform(0.5, 2.0) for _ in range(100)],
6     'velocidad': [random.uniform(5.0, 20.0) for _ in range(100)],
7     'temperatura': [random.randint(0, 400) for _ in range(100)]
8 }
9 df = pd.DataFrame(data)
10 df
```

	# masa	# velocidad	# temperatura
0	0.7503431160321467	6.076321364818335	299
1	1.2398932876851423	19.656629571959037	143
2	1.488390029122769	16.171448621103377	71
3	1.956155524500073	17.506552203032157	391
4	1.489561581723919	18.291977733516024	49
5	1.2366462354183316	11.11108678848975	153
6	1.4417990597330488	10.949313046789396	397
7	1.8753837443765222	6.78003822057714	214
8	0.9245960773704204	9.962501836977259	305
9	1.852290262443351	6.000278781723944	83

Discusión: Creamos un DataFrame con datos aleatorios para ilustrar la agrupación.

Manejo de datos con Pandas $\ni$ Agrupación y resumen de datos

```
1 # Crear una columna categórica
2 df['categoria'] = pd.cut(df['temperatura'], bins=[0,273,373,1000],
    ↳ labels=['Sólido', 'Líquido', 'Gaseoso'])
3
4 df
```

	# masa	# velocidad	# energia_cinetica	📦 categoria	
1		1.5	Missing value	82.6875	Alta
2		2.0	8.2	67.24	Baja
Fila 3		0.8	15.3	93.63600000000002	Alta

Discusión: Agrupar datos permite obtener estadísticas por categorías físicas (masas, temperaturas, etc.).

Manejo de datos con Pandas $\ni$ Agrupación y resumen de datos

```
1 # Agrupar y calcular estadísticas
2 df.groupby('categoria')['temperatura'].mean() # Puede ser .mean(), .std(), .sum(), etc.
```

categoria	# temperatura
Solido	127.55737704918033
Liquido	330.65625
Gaseoso	389.0

Discusión: Agrupar datos permite obtener estadísticas por categorías físicas (masas, temperaturas, etc.).

Manejo de datos con Pandas $\ni$ Información y Dimensiones de un DataFrame

```
1 # Operaciones básicas
2 df.info()           # Información del
    ↪ DataFrame
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  -
0   masa             100 non-null    float64
1   velocidad        100 non-null    float64
2   temperatura      100 non-null    int64
3   categoria        100 non-null    category
dtypes: category(1), float64(2), int64(1)
memory usage: 2.7 KB
```

```
1 # Operaciones básicas
2 df.shape           # Dimensiones
    ↪ (filas, columnas)
```

Output:

(100, 4)

Discusión: Estas operaciones básicas permiten inspeccionar rápidamente la estructura y dimensiones de un DataFrame, facilitando el análisis inicial de los datos.

Manejo de datos con Pandas $\ni$ Selección de Columnas importantes

```
1 # Seleccionar columnas
2 subset = df[['masa', 'velocidad']]    # Múltiples columnas
3 subset
```

	# masa	# velocidad
0	1.1622909903858079	6.294779128285901
1	1.2575392838565098	5.195686868016088
2	1.5875412989737128	12.157919722098546
3	1.2514011420766056	13.672540022650423
4	1.1538870112094566	16.95447565316892
5	1.4731483172445072	13.015316641086615
6	0.9072892094622428	12.942605710820246
7	0.74477645177605	18.15180828289533
8	1.4300533214655435	13.98126996753548
9	0.686840808473431	6.4791834216107

Discusión: La selección de columnas específicas permite centrarse en variables de interés, facilitando el análisis y la visualización de datos relevantes.

Manejo de datos con Pandas $\ni$ Otra forma de Selección y Filtrado de Datos

```
1 # Filtrado con condiciones
2 altas_temp = df[df['temperatura'] > 300]
3 altas_temp
```

# velocidad	# temperatura	📦 categoria
13.015316641086615	330	Liquido
7.80473808300243	368	Liquido
16.036247391526743	363	Liquido
14.312656446831161	382	Gaseoso
14.2988753285948	389	Gaseoso
19.257432372696933	306	Liquido
15.414132414861374	345	Liquido
5.531101211711803	362	Liquido
10.711592605859591	386	Gaseoso
16.888192119760735	363	Liquido

Discusión: El filtrado de datos permite identificar subconjuntos relevantes, como aquellos con temperaturas extremas, facilitando un análisis más enfocado.

Manejo de datos con Pandas $\ni$ Otra forma de Selección y Filtrado de Datos

```
1 # Filtrado múltiple
2 criteria = (df['masa'] > 1.0) & (df['velocidad'] > 10) & (df['categoria'] == 'Liquido')
3 filtered_df = df[criteria]
4 filtered_df
```

	# masa	# velocidad	# temperatura	📦 categoria
3	1.2514011420766056	13.672540022650423	284	Liquido
5	1.4731483172445072	13.015316641086615	330	Liquido
23	1.0930179125964166	18.015052688414393	295	Liquido
28	1.910202686031392	19.257432372696933	306	Liquido
49	1.0594663777168525	16.73561588787272	279	Liquido
64	1.473022133491637	12.222812720898194	353	Liquido
65	1.315947212724838	13.011127133650767	283	Liquido
69	1.2258166445419816	14.969849494919098	327	Liquido
81	1.800160284661923	19.9797030843529	277	Liquido
87	1.2158278235581306	17.6616430681612	331	Liquido

Discusión: El filtrado múltiple permite combinar varias condiciones para extraer subconjuntos específicos de datos, facilitando análisis detallados basados en múltiples criterios.

Manejo avanzado de datos con pandas

Manejo avanzado de datos con pandas $\ni$ Agrupación y Estadísticas Avanzadas

```
1
2 # Agrupar y calcular estadísticas
3 grouped = df.groupby('categoria').agg({
4     'masa': ['mean', 'min', 'max'],
5     'velocidad': ['mean', 'std'],
6     'temperatura': 'mean'
7 })
8
9 grouped
```

categoria	# ('masa', 'mean')	# ('masa', 'min')	# ('masa', 'max')
Solido	1.2729731138413485	0.5342694890797259	1.9549085906187986
Liquido	1.1562081797639387	0.5205567110184053	1.9103152773766008
Gaseoso	1.263349429207926	0.7893227110510436	1.8254734214937889

Discusión: La agrupación avanzada permite obtener múltiples estadísticas descriptivas por categoría, facilitando un análisis más profundo de los datos según diferentes criterios físicos.

Manejo avanzado de datos con pandas $\ni$ Correlaciones entre Variables

```
1 # Correlaciones entre variables
2 correlations = df[['masa', 'velocidad', 'temperatura']].corr()
3 print("Matriz de correlaciones:")
4 correlations
```

Matriz de correlaciones:

	# masa	# velocidad	# temperatura
masa	1.0	0.0012490296348494512	-0.008852259384265052
velocidad	0.0012490296348494512	1.0	0.13799237422100105
temperatura	-0.008852259384265052	0.13799237422100105	1.0

Discusión: Interpretación de la correlación

- Rango: $-1 \leq \rho \leq 1$.
- $\rho = -1$: correlación negativa perfecta — cuando una variable aumenta, la otra disminuye proporcionalmente.
- $\rho = 0$: ausencia de **correlación lineal** significativa.
- $\rho = 1$: correlación positiva perfecta — ambas variables aumentan proporcionalmente.

Limpieza de datos

Limpieza de datos $\ni$ Manejo de Datos Faltantes y Limpieza

```
1 import numpy as np
2 import random
3 # Simular valores faltantes (NaN) solo en la columna 'temperatura'
4 idx_temp = random.sample(range(100), 50) # Seleccionar 50 números aleatorios en una lista de 0 a 99
5 # idx_temp = [5, 12, 23, 34, 45, ... , 78, 89, 90] # Ejemplo
6 df.loc[idx_temp, 'temperatura'] = np.nan # Asignar NaN en esas posiciones o cualquier valor que
    ↪ quieras
7
8 df
```

	# masa	# velocidad	# temperatura	categoria
0	1.1622909903858079	6.294779128285901	8.0	Solido
1	1.2575392838565098	5.195686868016088	Missing value	Solido
2	1.5875412989737128	12.157919722098546	60.0	Solido
3	1.2514011420766056	13.672540022650423	Missing value	Liquido
4	1.1538870112094566	16.95447565316892	Missing value	Solido
5	1.4731483172445072	13.015316641086615	Missing value	Liquido
6	0.9072892094622428	12.942605710820246	34.0	Solido
7	0.74477645177605	18.15180828289533	Missing value	Liquido
8	1.4300533214655435	13.98126996753548	Missing value	Solido
9	0.686840808473431	6.4791834216107	186.0	Solido

Discusión: Simulamos datos faltantes en la columna de temperatura para ilustrar técnicas de limpieza.

Limpeza de datos $\ni$ Detectar Valores Faltantes

```
1 # Detectar valores faltantes
2 print("Valores nulos por columna:")
3 df.isnull().sum()
```

# 0	
masa	0
velocidad	0
temperatura	50
categoria	0

Discusión: Identificar valores faltantes es crucial para decidir cómo manejarlos en el análisis de datos.

Limpieza de datos $\ni$ Manejo de Datos Faltantes y Limpieza

```
1 # Opciones para manejar datos faltantes
2 df_dropped = df.dropna()           # Eliminar filas con NaN
3 df_dropped
```

	# masa	# velocidad	# temperatura	categoria
0	0.702945440225168	6.3865191775676955	109.0	Solido
1	0.9690671393777127	6.048542373822548	23.0	Solido
2	1.333310990489152	15.240299023778565	234.0	Solido
3	1.9231138793475124	19.90359211556003	353.0	Liquido
6	1.105603194347084	11.443547055237433	220.0	Solido
7	1.95634581987672	10.806381878743224	352.0	Liquido
10	1.3044770518799047	11.894737405806785	390.0	Gaseoso
14	1.3147638161813286	17.159520563824376	311.0	Liquido
17	0.6504724438774485	18.12720739180797	340.0	Liquido
18	1.1617941760020265	8.821518400931424	39.0	Solido

50 rows x 4 cols 10 per page << < Page 1 of 5 > >> 🔍 📊

Discusión: Eliminar filas con datos faltantes es una estrategia simple pero puede resultar en pérdida de información valiosa.

Limpieza de datos $\ni$ Manejo de Datos Faltantes y Limpieza

```
1 # Opciones para manejar datos faltantes
2 df_filled = df.fillna(df.mean())      # Rellenar con promedio
3 df_filled
```

```
TypeError: 'Categorical' with dtype category does not support reduction 'mean'
```

Discusión: Rellenar valores faltantes con la media es una técnica común, pero puede no ser adecuada si los datos no son homogéneos.

Limpieza de datos $\ni$ Manejo de Datos Faltantes y Limpieza

```
1 # Opciones para manejar datos faltantes
2 numeric_means = df.select_dtypes(include=[np.number]).mean()
3 numeric_means
```

# 0	
masa	1.2946635181576476
velocidad	12.879820643921464
temperatura	230.5

Discusión: Calculamos las medias solo de las columnas numéricas para evitar errores al rellenar.

```
1 # Rellenar solo columnas numéricas con sus medias
2 df_filled = df.fillna(numeric_means)
3 df_filled
```

	# masa	# velocidad	# temperatura	categoria
0	0.702945440225168	6.3865191775676955	109.0	Solido
1	0.9690671393777127	6.048542373822548	230.5	Solido
2	1.333310990489152	15.240299023778565	234.0	Solido
3	1.9231138793475124	19.90359211556003	230.5	Liquido
4	0.9942465857442007	12.41007460004106	230.5	Solido

Discusión: Esta técnica asegura que solo las columnas numéricas sean rellenadas con sus respectivas medias, evitando errores en columnas no numéricas.

Lectura y Exportación de Datos

Lectura y Exportación de Datos $\ni$ Lectura de Archivos y Exportación

Lectura de archivos: pandas facilita la importación de datos desde diversos formatos comunes en física y astronomía, como CSV y Excel.

```
1 # Leer diferentes formatos Ejemplos
2 df_csv = pd.read_csv('datos_experimento.csv') # CSV por defecto
3 df_excel = pd.read_excel('mediciones.xlsx', sheet_name='Hoja1') # Excel
4
5 # Parámetros útiles para CSV
6 df_custom = pd.read_csv('datos.csv',
7                          sep=';',          # Separador
8                          decimal=',',     # Separador decimal
9                          encoding='utf-8', # Codificación
10                         skiprows=2)       # Saltar filas
11
12 # Exportar resultados
13 df.to_csv('resultados_procesados.csv', index=False)
14 df.to_excel('analisis.xlsx', sheet_name='Datos')
15
16 # Exportar solo columnas específicas
17 df[['masa', 'velocidad']].to_csv('energia_datos.csv')
```

Ejemplo de archivo CSV: El siguiente es un ejemplo simple de un archivo CSV que contiene datos de masa, velocidad y temperatura. La primera fila contiene los nombres de las columnas, y las filas siguientes contienen los datos correspondientes, separados por comas.

datos.csv

masa,velocidad,temperatura

1.5 , 10.5 , 300

2.0 , 18.2 , 350

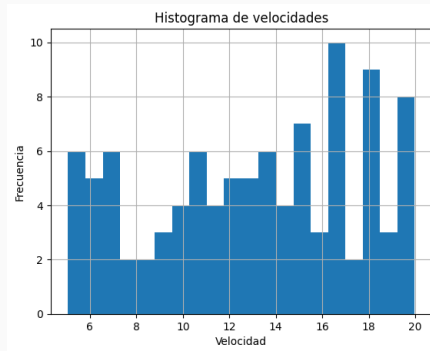
0.8 , 15.3 , 250

Discusión: El formato CSV es ampliamente utilizado para almacenar datos tabulares de manera sencilla y legible.

Visualización de Datos con Pandas

Visualización de Datos con Pandas $\ni$ Visualización rápida con pandas

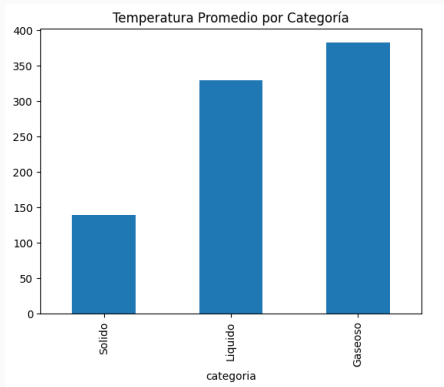
```
1 import matplotlib.pyplot as plt
2
3 df['velocidad'].hist(bins=20)
4 plt.title('Histograma de velocidades')
5 plt.xlabel('Velocidad')
6 plt.ylabel('Frecuencia')
7 plt.show()
```



Discusión: pandas integra métodos de visualización para análisis

Visualización de Datos con Pandas $\ni$ Visualización Integrada con Pandas y Matplotlib

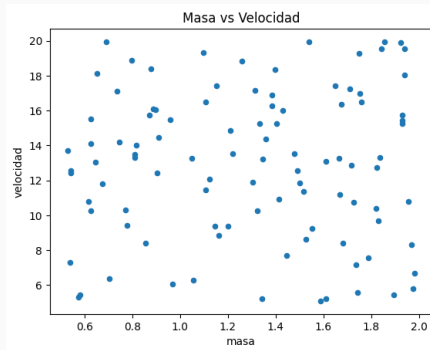
```
1 # Gráfico de barras por categorías
2 df.groupby('categoria')['temperatura'].mean().plot(kind='bar')
3 plt.title('Temperatura Promedio por Categoría')
4 plt.show()
```



Discusión: Este gráfico de barras muestra la temperatura promedio agrupada por categoría, facilitando el análisis comparativo.

Visualización de Datos con Pandas $\ni$ Visualización Integrada con Pandas y Matplotlib

```
1 import matplotlib.pyplot as plt
2 # Pandas tiene métodos de plotting integrados
3 df.plot(x='masa', y='velocidad', kind='scatter', title='Masa vs Velocidad')
4 plt.show()
```



Discusión: Este gráfico muestra la relación entre masa y velocidad en un scatter plot, útil para identificar patrones.

Ejercicios Guiados: Impactos de Meteoritos

Ejercicios Guiados: Impactos de Meteoritos $\ni$ Dataset base: Impactos de meteoritos (200 filas)

```
1 import numpy as np, pandas as pd
2
3 # Semilla y tamaño
4 rng = np.random.default_rng(42); n = 200
5
6 # Columnas base
7 diametro_m = rng.uniform(10, 1e5, n)           # 10 m a 100 km
8 densidad_kg_m3 = rng.uniform(1500, 8000, n)     # roca-carbonáceo-metálico
9 velocidad_kms = rng.uniform(11, 72, n)         # km/s
10 angulo_deg = rng.uniform(10, 80, n)            # grados
11 ids = [f"M{i:04d}" for i in range(1, n+1)]
12
13 df = pd.DataFrame({
14     "id": ids,
15     "diametro_m": diametro_m,
16     "densidad_kg_m3": densidad_kg_m3,
17     "velocidad_kms": velocidad_kms,
18     "angulo_deg": angulo_deg
19 })
20 df.head()
```

Nota física

Ley de cráter simplificada para Tierra (didáctica):

$$\text{crater_diam_km} \approx \frac{0.016}{1000} \cdot m^{1/3} \cdot v^{0.44} \cdot \sin(\theta)^{1/3} \text{ (resultado en km).}$$

Ejercicios Guiados: Impactos de Meteoritos $\ni$ Ejercicio 1:

Identificar el dataset



Enunciado

- Visualiza $\rightarrow$ `head()`, `tail()`, `sample(5)` y `df.columns`.
- Reporta unidades: diámetro (m), densidad (kg/m^3), velocidad (km/s), ángulo ($^\circ$).

Conceptos: Inspección inicial. **Física:** Revisión de magnitudes y unidades.

Ejercicios Guiados: Impactos de Meteoritos $\ni$ Solución 1 de Referencia:



Identificar el dataset

```
1 # Inspección inicial del DataFrame
2 df.head()
3 df.tail()
4 df.sample(5, random_state=0)
5 df.columns.tolist()
6
7 # Unidades:
8 # diametro_m [m], densidad_kg_m3 [kg/m^3],
9 # velocidad_kms [km/s], angulo_deg [°]
```

Discusión: Validamos estructura y unidades para evitar errores de interpretación.



Enunciado

- Ejecuta $\rightarrow$ `df.shape`, `df.info()` y `df.describe()`.
- Identifica columnas numéricas vs no numéricas.

Conceptos: Estructura y estadísticas rápidas.

Ejercicios Guiados: Impactos de Meteoritos $\ni$ Solución 2 de Referencia:

Dimensiones, tipos y describir

```
1 # Dimensiones, tipos y estadísticas
2 print("shape:", df.shape)
3 print("="*40)
4 df.info()
5 print("="*40)
6 print(df.describe())
7
8 # Columnas numéricas vs no numéricas
9 num_cols = df.select_dtypes(include='number').columns.tolist()
10 non_num_cols = df.select_dtypes(exclude='number').columns.tolist()
11 print("Numéricas:", num_cols)
12 print("No numéricas:", non_num_cols)
```

Discusión: Diferenciar tipos guía la limpieza y transformaciones posteriores.



Enunciado

- Filtra impactos con: `diametro_m > 1000`, `velocidad_kms > 20` y `30 ≤ angulo_deg ≤ 60`.
- Cuenta filas y calcula promedio de `velocidad_kms` del subconjunto.

Conceptos: Máscaras booleanas y selección.

Ejercicios Guiados: Impactos de Meteoritos $\ni$ Solución 3 de Referencia:

Filtrado físico

```
1 # Máscara booleana con tres condiciones
2 crit = (
3     (df['diametro_m'] > 1000) &
4     (df['velocidad_kms'] > 20) &
5     (df['angulo_deg'].between(30, 60))
6 )
7 sub = df[crit]
8 print("Filas filtradas:", len(sub))
9 print("Promedio velocidad_kms:", sub['velocidad_kms'].mean())
```

Discusión: El encadenamiento de condiciones permite filtros físicos realistas.



Enunciado

- Agrega: $\text{volumen_m3} = \frac{\pi}{6} \cdot \text{diametro_m}^3$.
- $\text{masa_kg} = \text{densidad_kg_m3} \cdot \text{volumen_m3}$.
- $\text{velocidad_ms} = 1000 \cdot \text{velocidad_kms}$.
- $\text{energia_J} = 0.5 \cdot \text{masa_kg} \cdot \text{velocidad_ms}^2$.

Conceptos: Operaciones vectorizadas y unidades.

Ejercicios Guiados: Impactos de Meteoritos $\ni$ Solución 4 de Referencia:



Masa y energía

```
1 import numpy as np
2
3 # Velocidad a m/s y masa (esfera):  $m = \rho * (\pi/6) * d^3$ 
4 df['velocidad_ms'] = df['velocidad_kms'] * 1000.0
5 df['masa_kg'] = df['densidad_kg_m3'] * (np.pi/6.0) * (df['diametro_m']**3)
6
7 # Energía cinética
8 df['energia_J'] = 0.5 * df['masa_kg'] * (df['velocidad_ms']**2)
9 df[['id', 'velocidad_ms', 'masa_kg', 'energia_J']].head()
```

Discusión: Cuidar la coherencia de unidades evita errores de órdenes de magnitud.

Ejercicios Guiados: Impactos de Meteoritos $\ni$ Ejercicio 5: Cráter (escala simplificada)



Enunciado

- Calcula θ en rad: `theta_rad = np.deg2rad(angulo_deg)`.
- Agrega `crater_diam_km` = $\frac{0.016}{1000} \cdot \text{masa_kg}^{1/3} \cdot \text{velocidad_ms}^{0.44} \cdot \sin(\theta)^{1/3}$.
- Reporta el top-5 por `crater_diam_km`.

Física: Ley de escala de cráter didáctica (Tierra).

Ejercicios Guiados: Impactos de Meteoritos $\ni$ Solución 5 de Referencia:



Cráter (escala simplificada)

```
1 import numpy as np
2
3 # Ángulo en radianes y diámetro de cráter
4 df['theta_rad'] = np.deg2rad(df['angulo_deg'])
5 df['crater_diam_km'] = (0.016 * (df['masa_kg']**(1/3)) * (df['velocidad_ms']**0.44) *
6   ↪ (np.sin(df['theta_rad']**(1/3))) / 1000.0
7 df.nlargest(5, 'crater_diam_km')[['id', 'diametro_m', 'crater_diam_km']]
```

Nota física

Ley de cráter simplificada para Tierra (didáctica):

$$\text{crater_diam_km} \approx \frac{0.016}{1000} \cdot m^{1/3} \cdot v^{0.44} \cdot \sin(\theta)^{1/3}.$$

Discusión: La fórmula ilustra tendencias cualitativas; no sustituye modelos detallados.

Ejercicios Guiados: Impactos de Meteoritos $\ni$ Ejercicio 6: Mortalidad por rangos de diámetro



Enunciado

- Define bins en metros: $[0,100)$, $[100,300)$, $[300,1000)$, $[1000,5000)$, $[5000,20000)$, $[20000,80000)$, $[80000, \infty)$.
- Etiquetas: *Inofensivo*, *Daños regionales*, *Ciudad devastada*, *Catástrofe regional*, *Catástrofe continental*, *Impacto global severo*, *Extinción total*.
- Crea `mortalidad` con `pd.cut` y muestra `value_counts()`.

Conceptos: Discretización con `pd.cut`.

Ejercicios Guiados: Impactos de Meteoritos $\ni$ Solución 6 de Referencia:

Mortalidad por rangos

```
1 import numpy as np, pandas as pd
2
3 bins = [0,100,300,1000,5000,20000,80000, np.inf]
4 labels = [
5     'Inofensivo', 'Daños regionales', 'Ciudad devastada',
6     'Catástrofe regional', 'Catástrofe continental',
7     'Impacto global severo', 'Extinción total'
8 ]
9 df['mortalidad'] = pd.cut(
10     df['diametro_m'], bins=bins, labels=labels, right=False, ordered=True)
11
12 print(df['mortalidad'].value_counts().sort_index())
```

Discusión: `pd.cut` permite análisis por categorías físicas interpretables.

Ejercicios Guiados: Impactos de Meteoritos $\ni$ Ejercicio 7: Agrupar y ranking por energía



Enunciado

- Agrupa por `mortalidad` y calcula media de `energia_J` y `crater_diam_km`.
- Obtén el top-5 impactos por `energia_J`.

Conceptos: `groupby` + `agg` y `nlargest`.

Ejercicios Guiados: Impactos de Meteoritos $\ni$ Solución 7 de Referencia:

Agrupar y ranking por energía

```
1 import numpy as np
2
3 # Asegurar columnas requeridas
4 df['velocidad_ms'] = df['velocidad_kms'] * 1000.0
5 df['masa_kg'] = df['densidad_kg_m3'] * (np.pi/6.0) * (df['diametro_m']**3)
6 df['energia_J'] = 0.5 * df['masa_kg'] * (df['velocidad_ms']**2)
7
8 df['theta_rad'] = np.deg2rad(df['angulo_deg'])
9 df['crater_diam_km'] = (
10     0.016 * (df['masa_kg']**(1/3)) * (df['velocidad_ms']**0.44) * (np.sin(df['theta_rad'])**(1/3))
11 ) / 1000.0
12
13 # Resumen por categoría y top-5 por energía
14 res = df.groupby('mortalidad')[['energia_J', 'crater_diam_km']].mean()
15 print(res)
16 print(df.nlargest(5, 'energia_J')[['id', 'energia_J', 'crater_diam_km']])
```

Discusión: groupby+agg resume por severidad; nlargest identifica eventos extremos.



Enunciado

- Dibuja histograma de `velocidad_kms` y scatter `diametro_m` vs `crater_diam_km` coloreado por `mortalidad`.
- Exporta `df` limpio a CSV (`impactos_meteoritos.csv`).

Conceptos: visualización rápida y persistencia con `to_csv`.

Ejercicios Guiados: Impactos de Meteoritos $\ni$ Solución 8 de Referencia:

Visualización y exportación

```
1 import matplotlib.pyplot as plt
2
3 # Histograma de velocidad
4 df['velocidad_kms'].hist(bins=20)
5 plt.xlabel('Velocidad [km/s]')
6 plt.ylabel('Frecuencia')
7 plt.title('Histograma de velocidad')
8 plt.show()
9
10 # Asegurar categoría 'mortalidad'
11 import numpy as np, pandas as pd
12 bins = [0,100,300,1000,5000,20000,80000, np.inf]
13 labels = ['Inofensivo', 'Daños regionales', 'Ciudad
    ↳ devastada', 'Catástrofe regional', 'Catástrofe
    ↳ continental', 'Impacto global severo', 'Extinción
    ↳ total']
14
15 df['mortalidad'] = pd.cut(df['diametro_m'],
    ↳ bins=bins, labels=labels, right=False,
    ↳ ordered=True)
```

```
17 # Scatter coloreado por categoría
18 colors =
    ↳ df['mortalidad'].astype('category').cat.codes
19 plt.scatter(df['diametro_m'], df['crater_diam_km'],
    ↳ c=colors, cmap='viridis', s=12)
20 plt.xlabel('Diámetro [m]'); plt.ylabel('Diám.
    ↳ cráter [km]')
21 plt.title('Diámetro vs Cráter (color =
    ↳ mortalidad)'); plt.show()
22
23 # Exportar CSV
24 df.to_csv('impactos_meteoritos.csv', index=False)
```

Discusión: Visualizamos distribución y relación; persistimos resultados para análisis externo.